FoodLink: A Coordination Platform for Surplus Food Redistribution

Project Proposal for UCS503P
Submitted to: Dr. Raghav B. Venkataramaiyer
Thapar Institute of Engineering and Technology

Aayushi Sud, CSED* Rishi Garg, CSED† Yugam Heer, CSED‡
Himanshi Singla, CSED§

August 31, 2026

1 Higher-order goal

. . . secondary framing

This project aims to reduce avoidable food waste and improve local food security by shortening the time and effort required to move edible surplus food from where it is produced to organisations that can serve it. While hunger and food waste are far broader than any single campus or city, the immediate engineering objective is to translate *timely surplus redistribution* into a measurable, deployable software solution.

2 Time-to-value

. . . primary heuristic

- We will deliver meaningful increments quickly by prototyping the core donation-to-delivery workflow and validating it with real donors and recipient organisations within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations (and targeting CI-backed automation early) to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration, then improved based on user feedback.

3 Problem Statement

Institutional kitchens in a tier-2/3 Indian city — college messes, hostel canteens, bakeries, banquet halls — regularly finish a service with edible food left over, while nearby shelters, community kitchens and NGOs are simultaneously short of meals. The two sides rarely connect in time. Common pain points include:

*Roll No: 1024030906, Email: asud_be24@thapar.edu
†Roll No: 1024030909, Email: rgarg8_be24@thapar.edu
‡Roll No: 1024030956, Email: yheer_be24@thapar.edu
§Roll No: 1024031166, Email: hsingla1_be24@thapar.edu

- **A hard perishability deadline:** cooked surplus is safe to serve for only a few hours. Coordination conducted over phone calls and ad-hoc messaging routinely outlasts that window, after which the food must be discarded.
- **Fragmentation:** a donor contacts whichever organisation it happens to know. There is no shared pool, so availability is invisible to everyone else.
- **Low transparency:** a donor seldom learns whether the food was actually collected and served; a recipient cannot see what is available right now.
- **Last-mile coordination cost:** unlike information, food must be physically moved. Volunteer availability is informal and untracked, so a willing donor and a willing recipient can still fail to complete a handover.
- **Insufficient information for a safe decision:** a recipient needs to know what the food is, when it was prepared, how it has been stored, and by when it must be collected — details that are lost in informal channels.

Why this matters now (contextual relevance): On and around a campus, the binding constraint is coordination rather than supply. Surplus already exists and recipient organisations already exist; a modest reduction in coordination delay converts food that would have been discarded into served meals.

4 Proposed Solution *... Software Proposition*

4.1 Overview

Build a **web-based** "Surplus-to-Served" platform serving four roles — donor, recipient organisation, volunteer and administrator — with the following components:

- **Donor posting portal:** a donor posts surplus with food type, quantity, preparation time, storage condition, pickup location and a pickup deadline (photo optional).
- **Assisted matching:** the system ranks eligible recipient organisations against the posting on transparent criteria and surfaces the strongest candidates, together with the reasons for the ranking.
- **Volunteer dispatch:** an accepted donation becomes a pickup task that a volunteer can claim, with pickup and drop-off details.
- **Status transparency:** one tracked lifecycle per donation, visible to every party involved, with notifications on each transition.
- **Admin/ops dashboard:** performance summaries (meals redistributed, time to claim, donations lost to deadline expiry, current backlog).

4.2 Core workflow

1. A donor posts surplus food and immediately receives a donation ID; the pickup deadline is recorded at this point.
2. The system ranks eligible recipient organisations and notifies the strongest candidates; the posting also enters a pool visible to all verified recipients.
3. A recipient organisation reviews the posting and accepts it. The system never assigns food to a recipient without that explicit acceptance.

4. The accepted donation becomes a pickup task; a volunteer claims it (or the recipient elects to collect the food itself).
5. The volunteer updates status through the handover: Picked Up → Delivered.
6. The recipient confirms receipt, closing the donation as Completed. A donation that is not claimed before its deadline is closed as Expired and recorded as a measured loss.

4.3 Operational constraints for an educational, tier-2/3 setting

- Keep the solution usable on low-to-mid range devices via responsive web design. Donors post from a working kitchen and volunteers act while travelling, so the interface must be mobile-first rather than merely mobile-tolerant.
- Avoid dependence on advanced research algorithms; focus on workflow, automation, and system correctness.
- Design for deployability using common web hosting and managed databases.
- Treat the platform as a coordination tool, not a food-safety authority: it records and displays the information a recipient needs in order to decide, and the recipient inspects the food at handover.

5 Solution Approach *... Engineering focus*

This is an engineering course project, so we focus on scalable administrative and workflow aspects rather than novel algorithm research.

5.1 Match assistance *... non-research*

Recipient ranking will be heuristic-based and fully explainable:

- Score each eligible recipient on a small set of normalised criteria — travel distance, requested quantity against donated quantity, storage compatibility, time remaining before the pickup deadline, and historical completion reliability.
- Combine them using fixed, published weights (a transparent weighted-sum model) to produce a 0–100 suitability score, without claiming state-of-the-art performance.
- Show the top candidates and the per-criterion reasons to the recipient and to the administrator; the recipient always makes the final decision and the system never auto-assigns or auto-rejects a donation.
- Keep the scoring behind a stable interface so that the heuristic can later be replaced by a learned ranker without changes to the surrounding workflow.

5.2 Web architecture *... web-based solution*

A typical 3-tier web architecture:

- **Frontend:** responsive role-specific UI for donors, recipients, volunteers and administrators (posting form, available-donations pool, pickup tasks, status timeline, dashboard).
- **Backend API:** authentication, donation posting, recipient ranking, status transitions, volunteer assignment, deadline expiry.

- **Database:** store users, organisations, donations, status history, volunteer tasks and attachment metadata.

All status transitions and their timestamps are recorded server-side, since the evaluation metrics in Section 6 are derived from them.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests on every change.
- Produce repeatable deployment artifacts to staging.
- Enable safe and frequent releases of incremental features.

6 Evaluation Criterion

. . . measurable and attributable

We define success using measurable metrics tied to the core workflow. Because surplus food expires, our metrics are framed around speed and completion rather than volume alone.

6.1 Primary evaluation metric

Time-to-claim (TTC): time between a donation being posted and a recipient organisation accepting it.

- Target: median TTC $\leq$ 45 minutes during the pilot window.
- Attribution: measured from database timestamps (posting creation vs. the acceptance transition), with no manual entry.
- Rationale: TTC is the delay the platform is designed to remove, and it is the delay that decides whether the remaining steps can finish before the food expires.

6.2 Secondary evaluation metrics

- **Rescue rate:** percentage of posted donations that reach Completed before their stated pickup deadline. Target $\geq$ 80% during the pilot.
- **Expiry loss rate:** percentage of postings closed unclaimed at deadline — the direct counter-metric to rescue rate.
- **Handover time:** time from acceptance to delivery, isolating last-mile volunteer performance from matching performance.
- **User clarity:** donor- and recipient-reported clarity of donation status using a simple 1–5 feedback question.
- **Reliability:** availability of staging/production for login and posting during service hours (e.g., $\geq$ 99% uptime in pilot).

6.3 Pilot validation plan *. . . high-level*

- Recruit 2–3 campus donors (mess or canteen operators), 3–5 recipient organisations, and 8–12 student volunteers (roles simulated within the team where a real counterpart is unavailable).
- Establish the baseline for TTC and rescue rate through short interviews on the current phone/messaging practice, since the existing process leaves no records to mine.
- Compare metrics before/after within the iteration window using the timestamps logged by the system.

7 Scalability *. . . with foresight*

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in number of donations, participating organisations and concurrent dashboard usage by:
   - decoupling components (frontend/backend),
   - enabling pagination and indexing for common queries (open donations by locality and deadline),
   - using stateless API design,
   - bounding the ranking computation to recipients within a serviceable radius rather than the full set.
2. **Operationally deployable (practically):** The system should run on common web hosting:
   - managed database,
   - containerized or platform-hosted deployment,
   - CI/CD pipeline for staging/production.

8 Engine availability heuristic

A mature set of "engines" (tooling/services) is available to implement this as a web-based project:

- Web framework for REST APIs and templated/reactive pages.
- Managed relational or document database.
- Authentication approach (token-based or session-based) with role-based access for the four user types.
- Object storage for optional food photographs (or local storage for pilot).
- Geocoding and straight-line distance computation over stored coordinates; no routing research is required for the pilot.
- Scheduled job execution for deadline expiry and reminders.
- Test framework for unit/integration tests.
- CI runner and staging deployment target.

We will use these mature components to focus on workflow design, correctness, and measurement rather than inventing new infrastructure.

9 Project scope and deliverables *. . . iteration-friendly*

9.1 Initial deliverable *. . . first iteration*

- Donor posting form + donation ID generation, with pickup deadline captured at posting
- Recipient workflow: view the open-donation pool and accept a donation
- Volunteer workflow: claim a pickup task and update status through delivery
- Shared status page: view the donation timeline/history across all roles
- Basic logging and server-side timestamps for TTC and rescue-rate measurement
- CI: build + backend unit tests + linting
- CD to staging with a single command or pipeline job

9.2 Subsequent deliverables

- Ranked recipient suggestions with per-criterion explanations shown to the recipient
- Standing requirement postings by recipient organisations, so demand is visible before surplus appears
- Notification layer (email/SMS optional in a later iteration; can start with in-app notifications)
- Automatic deadline expiry with loss accounting
- Admin dashboard for metrics and backlog
- Optional food photographs on postings
- Improved search/filtering and indexed queries for scale readiness

10 Risks and mitigations

- **Food safety and liability:** the platform coordinates handovers and does not certify food. We record preparation time, storage condition and deadline, display them before acceptance, and leave inspection to the recipient at handover. The system makes no health or safety guarantee.
- **Perishability outrunning coordination:** enforce the pickup deadline server-side, surface remaining time prominently, and close expired postings automatically so that losses are measured rather than hidden.
- **Volunteer unavailability breaking the last mile:** allow recipient self-pickup as a first-class fallback so that a match is not wasted when no volunteer claims the task.
- **Donor and staff adoption:** keep the posting form short enough to complete during cleanup; keep status updates to single-tap actions.
- **Evaluation measurement ambiguity:** instrument timestamps and define clear status transitions before development begins.
- **Data privacy / consent:** minimize collected data; reveal contact phone numbers only after a match is accepted; keep photographs optional; store only what is needed.
- **Operational issues in pilot:** use staging early; ensure CI/CD produces repeatable deployments.

11 Summary

This project proposes a deployable web platform that connects institutional food surplus with recipient organisations before that surplus expires, in a tier-2/3 city and campus context. It meets the course criteria by emphasizing time-to-value through rapid iterations, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially time-to-claim and rescue rate). It remains focused on engineering workflow and scalability readiness rather than research-driven novelty: recipient ranking is a transparent, explainable heuristic with a human decision at every match.